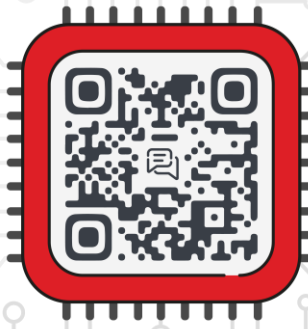




1

12

1440 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

OOP Overview & Enumeration

د. محمد الأحمد



RB Informatics; 09/10/2024

برمجة ٣

بسم الله الرحمن الرحيم

أعزاءنا الطلبة و مهندسي المستقبل نبدأ بمقرر البرمجة (3) راجين من الله تعالى أن
تعم الفائدة و النفع لنا و لكم

سندرس في مقرر البرمجة (3) الـ Advanced Java و ننوه الى أن أسئلة الامتحان ستكون جميعها باللغة
الإنكليزية كما أنه يجب تطبيق الأفكار عمليا
سنركز في محاضرتنا على :

Overview opp (1)

Enumeration Type in Java (2)

تعريف الـ opp (Object Oriented Programming) :

Set of related object مجموعة من الأغراض المتفاعلة مع بعضها

أولا : مراجعة اللغة غرضية التوجه oop:

ترتكز اللغة غرضية التوجه على ثلاث مبادئ و اذا تحققت هذه المبادئ في أي لغة فنستطيع القول عنها أنها oop :
وهي :

Encapsulation (1)

Polymorphism (2)

Inheritance (3)

في معظم المراجع يتم إضافة مبدأ رابع وهو Abstraction ولكن هذا خاطئ لأنه فعليا هو طريقة تفكير
نقوم بها بأي لغة و لكن بطرق مختلفة

▪ Abstraction : هي عملية (Process) يقوم بها العقل للتركيز على جوانب أساسية و إهمال التفاصيل أو أشياء أخرى و هي تختلف من لغة الى أخرى (طريقة تفكير)

▪ لنلاحظ أنه لحل أي مشكلة يجب فهم المشكلة و

تجميع Data و تعريفها
مجموعة من الـ Function

و الان سنقارن طريقة الـ Abstraction في مثال البنك في اللغة الإجرائية و اللغة غرضية التوجه

غرضية التوجه oop

```
Class account {
Account number;
float balance;
deposit();
withdraw();
check Balance ();
}
```

❖ في اللغة غرضية التوجه نفكر بالمشكلة على أنها أغراض ونقسمها الى كلاسات وخلق obj

الإجرائية pop

```
balance;
deposit();
check balance();
withdraw();
```

❖ في اللغة الإجرائية نفكر بـ Global Data ومجموعة من الـ Function لتقديم الخدمات

❖ في مثال الطالب يهمنا معرفة اسم الطالب ورقمه و معدله ولكن لا يهمنا لون عيونه فهنا استخدمنا الـ Abstraction بحيث ركزنا على الجوانب الأساسية (الاسم ، الرقم ، المعدل) و أهملنا التفاصيل

▪ ملاحظة: لغة C و باسكال ليست لغات غرضية التوجه لأنها لا تحتوي على مفهوم الـ class

دعونا نتعرف على مفهوم الـ class و object :

▪ Class :

هو Blueprint (مخطط) يتولد منه مجموعة من الـ instance (نسخ) وهي ما يسمى بـ الـ object و أيضا يمكن تعريف الـ class على انها Type أو user defined type / data type هناك كلاسات جاهزة في اللغة ضمن package تسهل للمبرمج الاستخدام مثل :

▪ Java.lang , java.util , java.io , java.Api

▪ فائدة أريسيزية : الـ java.Api تحتوي على العديد من الحزم و الكلاسات الجاهزة مثل , string , ArrayList , file

■ object :

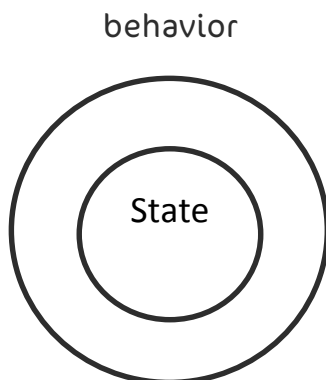
- هي نسخة من الكلاس الذي له وجود حقيقي و هي التي تحجز مكان في الذاكرة Instance class
- نسمي عملية خلق ال obj ب instantiation

Encapsulation (التغليف)

➤ هي حزم أو ربط ال Data مع ال Method التي تقوم ب implement لهذه ال Data في بنية واحدة وهي ال class و هذا ما يقوم بحماية ال Data حيث لا يمكن الوصول لها الا عن طريق هذه ال Method بالإضافة الى ال Access Modifiers التي تضمن ال security

➤ ما الهدف من ال Encapsulation ؟

إخفاء ال Data و حمايتها (Data Security)



نلاحظ أن ال State = variable
وال behavior = Method

➤ في اللغات القديمة كانت ال Data مكشوفة و Free نعرفها كما نشاء أما في ال oop فان ال Data هي موجودات أصلية (Real object)

تذكرة ب Access Modifiers :

Private
Protected
Default
public

أكثر أمناً

أقل أمناً

- بشكل عام يجب أن تكون الـ data هي Private و نصل اليها عن طريق الـ set و الـ get و الـ method نعرفها public لأنها خدمات و نادراً ما تعرف الـ data كـ public و اذا أردنا استخدام مفهوم الـ inheritance نعرف الـ data كـ protected

ماذا نضع ضمن الـ class ؟

```
Class A {
    Private Data;
    Constructors
    Set & get
    Methods
}
```

أولاً : Constructor

- تعمل هذه الـ Method فقط لحظة خلق الـ obj ، عند إضافة الـ new يمدد المتحولات (إعطاء القيم الابتدائية) و اذا لم يوجد constructor يعمل الـ constructor الافتراضي و يعطي القيم

```

(0) ← قيم عددية
false ← bool
null ← reference
```

و يمكننا تعديل القيم لاحقاً باستخدام الـ set و الـ get

ثانياً: مجموعة من الـ set و الـ get من النمط public

- و هي بوابة الوصول للـ data وفق ما يسمى بمبدأ الصقل المتدرج أي نبدأ حل المشكلة خطوة بخطوة بالتدريج **مثل :** نعرف class student و نزيد رقمه و معدله (أي يتطور الحل بشكل تزايد)

ثالثاً: services

- الكلاس بدون ميثودات تقدم خدمات ليس له أي معنى
- **ماهو الـ package ؟**
- هي حاوية تجمع كل الكلاسات و الانترفيسات ضمن بيئة واحدة
- يمكننا استخدام الـ file المحتواة ضمن الـ package خارج الـ package بواسطة تعليمة import
- الـ package يحقق مفهوم الـ Encapsulation

مثال : انشاء مجلد ، نسميه برمجة(3) وفيه مجلدات أيضاً عملي و نظري فهذا شبيهه بالـ package

➤ لادراج كلاس معين : `import package.class`

اسم الـ package

اسم الـ class

(تعددية الأشكال) polymorphism

➤ هناك علاقة بين الـ polymorphism و الـ inheritance

➤ تعددية الأشكال : أي أن الـ method و الـ obj يمكن أن تقدم أكثر من خدمة أي أنه نفس الكائن ولكن بسلوك مختلف

أنواع الـ polymorphism:

1. Dynamic polymorphism

يحدث أثناء الـ runtime يطبق باستخدام الـ method overriding (إعادة التعريف) ، و هو مرتبط بالوراثة لان الـ override تستخدم عند الوراثة

■ تذكرة :

الـ method override تحدث عندما يرث class الابن من class الاب و تقوم بتعريف الـ method مرة أخرى في كلاس الابن و التعديل في سلوكها و يكون للميثود في الكلاسين ذات التوقيع و فقط يختلف السلوك (التعليمات) داخل الميثود

2. Static

يحدث أثناء الـ compile time يطبق باستخدام الـ method overloading (التحميل الزائد) **مثال :**

```
Class A
Int f1(int a);
Int f1(float a, float b);
}
```

تذكرة :

- method overloading يكون لعدة method ذات الاسم ولكنها تختلف بالتوقيع signature و ذلك فقط باختلاف ال parameter (عدد البارامترات او ترتيبها او نوعها)

فائدة: ال overload تسهيل للمبرمج

لا يهم ال return type لأن المسؤول هو ال compiler و ال return type يحصل أثناء التغيير

Inheritance

➤ هي علاقة Is a

➤ هي العلاقة بين

Specification

كلاس الابن

sub class

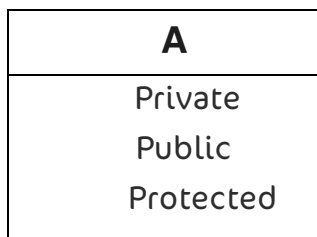
9

Generalization

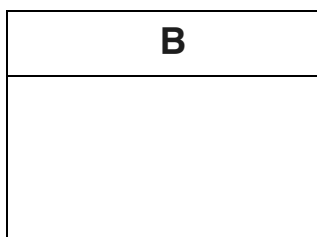
كلاس الأب

Super class

- أي علاقة هرمية هي علاقة وراثة ، فمثلاً ال person و ال teacher هي علاقة وراثة ، فكل ال person هو الأب (عام) و كلاس ال teacher هو الابن لأنه مختص ، فالابن يرث من الأب صفات عامة ثم يضيف عليها فيرث الابن كل methods الأب و بإمكانه أن يعرفها مجدداً عن طريق ال overloading
- نمثل أي كلاس بالـ UML بمستطيل



→ Super



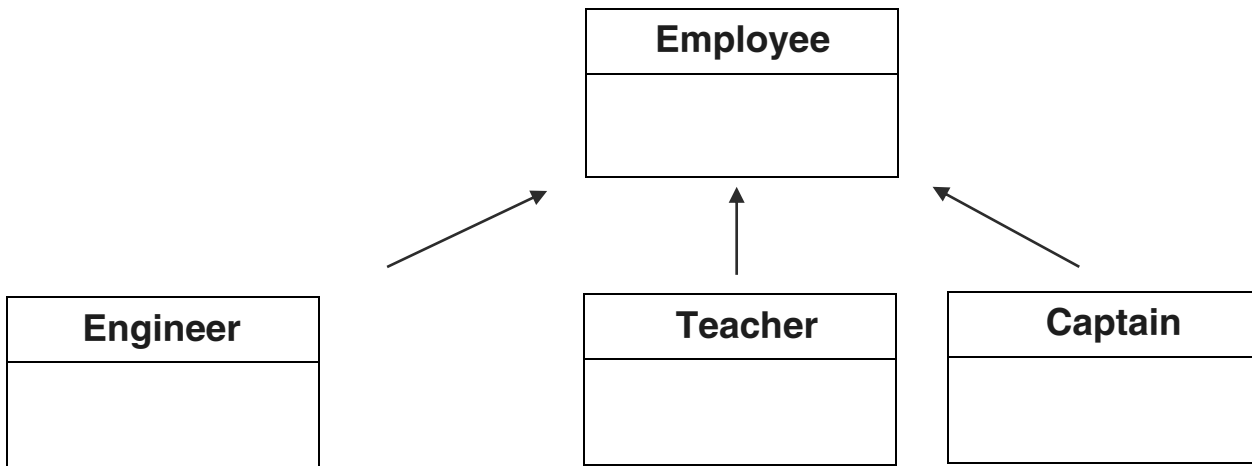
→ Sub



■ ملاحظة : أحياناً هناك method في الأب لا تنفع عند الابن فيمكن إعادة تعريفها بال body للابن ولا يجوز تغيير ال head للميثود و اذا تغير يصبح method overloading

فائدة : في لغة c++ يسمى الكلاس الابن بـ derived class أما في جافا فيسمى بـ sub class

مثال :



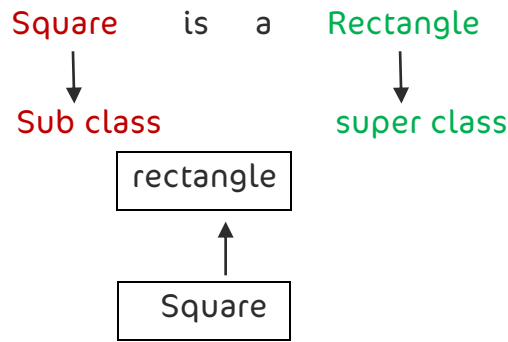
- في كلاس الأب هناك ميثود وهي calculation salary() ستورث هذه الميثود للأبناء و تكون طريقة حساب الراتب للمعلم و المهندس واحدة ولكنها تختلف بالنسبة للطيار فنعيد تعريف هذه الميثود وفق ما يناسب
- ما هو الهدف من الوراثة ؟
- القدرة على إعادة الاستخدام reusability

فائدة :

الأفضل عدم استخدام الوراثة بعمق كبير لأنه يؤثر على أداء البرنامج ، فيجب أن تكون الوراثة متوازنة و يجب أن يكون الكود المكتوب قابل للصيانة والتوسع وإعادة الاستخدام

والآن سنعطي مثال عن وراثة قد تبدو صحيحة ولكنها ستضج لاحقاً انها وراثة خاطئة:

- لدينا في الرياضيات قاعدة تقول أن كل مربع هو مستطيل وهي صحيحة رياضياً ولكن عند تحويلها الى كود برمجي سيكون خاطئ



■ تنويه : حسب قانون ليسكوف سيتضح أنها وراثة خاطئة و بإمكانهم الاطلاع و البحث عن ذلك أكثر

وبذلك نكون قد أنهينا مراجعة المبادئ الثلاثة في الـ oop

و الآن دعونا نتذكر أنواع الكلاسات في java :

■ Concrete class :

نستطيع خلق زب من هذا الكلاس باستخدام الـ keyword (new)

■ Abstract class :

-ممنوع خلق زب منه

-لا يمكن استخدام الـ keyword new معه

-و يمكننا أن نجعل الـ class نوعه abstract إما :

← abstract keyword

← وجود abstract method واحدة تكفي لجعل كلاس ما هو abstract class

■ الهدف من الـ abstract class دعم الوراثة

■ ملاحظة (1): الـ abstract method هي ميثود معرّفة في الكلاس الأب ويجب توريثها للأبناء و يجب القيام بـ implementation لها في جميع الأبناء و ليس لها body

■ ملاحظة (2): الـ pure function في c++ تقابلها الـ abstract function في الجافا

Interface

- Full abstract class تدعم الـ multi inheritance
- في الجافا ممنوع أن يرث interface من عدة كلاسات فقط كلاس واحد
- ← جميع الـ data تكون public static final
- ← جميع الميثود تكون public abstract

هناك نوعين من الـ function, variable :

Instance

أي تابع أو متحول بدون static

Static

أي تابع أو متحول تسبقه keyword static

في التوابع

تستدعى بدون obj (باسم الـ class)

في المتحولات

يعرف على مستوى الكلاس و يحجز مكان واحد
في الذاكرة لهذا المتحول ويكون مشترك بجميع

الـ obj

- الهدف من الـ static الحفاظ على الذاكرة

Final

variable

يصبح ثابت

method

لا يمكن أن تقوم بـ

override معها

Class

يصبح الكلاس لا يورث

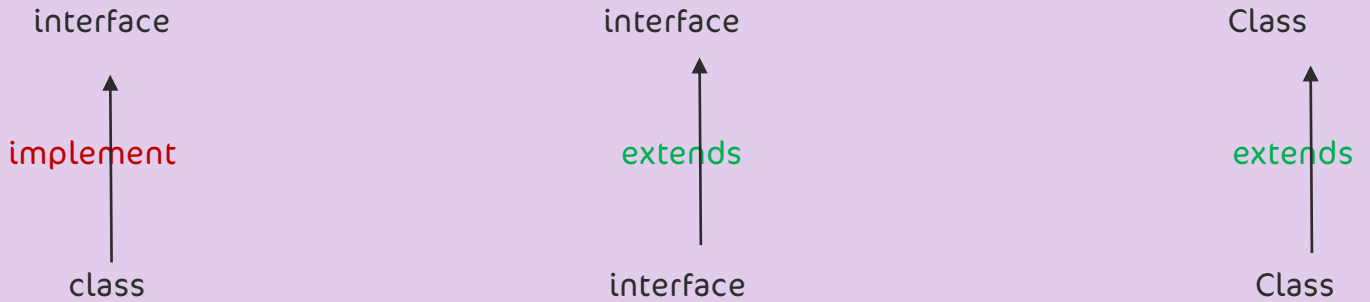
ولكن يمكن خلق obj منه

■ ملاحظة : في c++ نستخدم للثوابت const في الـ java نستخدم final

ملاحظة (2) :

- نستخدم بين شيئين متماثلين extends و بين class و interface نستخدم implements

- يمكن أن نقوم بـ implements لأكثر من interface



Enumeration in java

النمط التعدادي : هذا النمط لم يكن موجوداً في اللغة و في java.s تم اضافته

- Enum: special class of fixed set of constants

أي مجموعة من الثوابت (المتغيرات الغير قابلة للتغيير) و هو مخطط يحوي predefined constants

■ مثال :

أيام الأسبوع ، الأشهر ، الفصول ، مقاييس الألبسة ، الألوان
و هذه القيم الثابتة تقابل قيم صحيحة (لها ترتيب)

-لانشاء الـ enum نستخدم الكلمة المفتاحية enum في مكان كلمة class و نفصل بين الثوابت بـ (,) فاصلة عادية

ملاحظة : من الممكن وضع (;) في نهاية القيم الثابتة و اذا لم نضعها لا يعطي الـ compile error

■ مثل :

```
enum RGB{RED, GREEN;}
```

-ينصح بكتابة الثوابت بـ upper case letters

-على الرغم من أن القيم الثابتة التي بداخل الـ enum يقابلها أعداد صحيحة الا أننا نستخدم اسم القيمة الثابتة و ذلك للتسهيل على المستخدم

■ مثال :

```
enum color { RED , GREEN ;}
```

فنلاحظ أن الـ RED → int final=0

GREEN → int final=1

الا أننا استخدمنا القيمة العددية الصحيحة ، وهذه القيم الصحيحة هي `public static final` أي تحجز مكان في الذاكرة لمرة واحدة

- يطلق على الـ `enum` مفهوم الـ `type safe` أي أنه يمنع ادخال قيم غير متوافقة مما يجعل البرنامج أكثر اماناً ودقة ، فعلى سبيل المثال عند استخدام `enum` لتمثيل أنواع مختلفة من السيارات مثل `SUV` , `BMW` فان `enum` سيضمن أنه يمكن فقط تمثيل هذه القيم المحددة ولا يمكن استخدام قيم أخرى غير معرفة مسبقاً

- التعامل مع الـ `enum` يشبه التعامل مع الـ `final static class` حيث لا يمكن انشاء `obj` منه ولا نستخدم معه `new`

- ويمكن الوصول بشكل مباشر للثوابت الموضوعة فيه:

```
color c = color.RED;
System.out.print(c);
```

- الـ `enum` هو كلاس `by default` وهو كلاس داخلي ولا يمكن أن يرث من أي كلاس آخر لأنه ضمناً يرث من كلاس اسمه `enum` (موجود في `java.lang.enum`)

- لا داعي أن يتم استدعاؤها بواسطة `import` و إنما يمكن أن يقوم بـ `implement` لـ `interface` أو أكثر

- يمكن تعريف الـ `enum` داخل أي كلاس أو في ملف لوحده أو تعريفه في الـ `main class` و استخدامه هناك

- يكون الكونستروكتر من النمط `private` لذلك لا يمكن انشاء `instance` منه و اذا اردنا تعريف كونستركتور يجب تعريفه `private`

لماذا نسميه بالنمط التعدادي ؟

لأن كل ثابت له `index` ابتداءً من الصفر و يتم معرفة الـ `index` الخاص به باستخدام تابع `ordinal`

➤ استخدام الـ `enum` مع الـ `switch`

```
1 enum RGB { RED , GREEN ,BLUE;}
2 class Test {
3     public static void main (string argument) {
4         RGB c = RGB.RED ;
5         Switch (c){
6             case RED :
7                 System.out.print(" RED");
8                 break;
9             case GREEN:
10                System.out.print("GREEN");
11                Break;}}}
```

➤ الاستخدام مع الحلقات :

```
for( RGB c : RGB.values ) {  
    System.out.print(c);  
}
```

➤ نتابع الـ values للحصول على مصفوفة من نوع الـ enum و فيها نسخة من كل قيمة

مثال اريبيسيزي :

استخدام تابع الـ ordinal مع الـ enum :

```
1  enum Day { SUNDAY, MONDAY, TUESDAY,  
WEDNESDAY, THURSDAY, FRIDAY, SATURDAY }  
2  
3  class Test {  
4      public static void printordinal(Day day) {  
5          System.out.println("the ordinal of" + day + "is  
: " + day.ordinal());  
6      }  
7  
8      public static void main(String[] args) {  
9          Day Friday = Day.FRIDAY;  
10         printordinal(Friday);  
11     }  
12 }
```

(The End)